

Pametni asistent za društvenu igru Splendor zasnovan na sistemu pravila

Dušica Pešić sv46/2022

1. Motivacija i opis problema

Društvena igra **Splendor** je **strateška engine-building igra** u kojoj igrači skupljaju dragulje, kupuju razvojne karte i pokušavaju da prvi dostignu 15 prestižnih poena. Iako su osnovna pravila igre jednostavna, kvalitet odluke u svakom potezu zavisi od više povezanih faktora: trenutnih žetona, dostupnih karata, trajnih bonusa, rezervisanih karata, plemića, faze igre i mogućih poteza protivnika.

Igrač u svakom potezu mora da proceni da li je bolje:

- uzeti žetone,
- kupiti dostupnu kartu,
- rezervisati kartu,
- kupiti rezervisanu kartu,
- fokusirati se na plemića,
- ubrzati završnicu,
- ili blokirati protivnika.

Ovakva struktura igre je pogodna za primenu sistema baziranih na znanju, jer odluke igrača često počivaju na **strategiji sa heurističkim pravilima**.

Motivacija projekta je **razvoj pametnog asistenta koji pomaže igraču da izabere kvalitetan potez**. Sistem rangira moguće poteze i daje preporuku i objašnjenje zašto je određeni potez izabran. Ovaj asistent je namenjen igri za **2 igrača**.

Napomena: Sistem ne obrađuje izbor žetona koje treba vratiti ukoliko igrač prekorači limit od 10 žetona. Takođe, pri davanju preporuke ne razmatra rezervaciju nepoznate karte, već zaključuje isključivo na osnovu trenutno poznatih i vidljivih karata.

2. Pregled igre

U igri Splendor igrač skuplja **prestižne poene** kupovinom razvojnih karata i privlačenjem plemića. Tokom poteza može da izabere jednu od tri osnovne akcije:

- **uzimanje žetona**,
- **rezervisanje** razvojne karte,
- **kupovina** razvojne karte.

Igrač može uzeti tri različita žetona ili dva ista, ako ih u banci ima dovoljno. Na kraju poteza ne sme imati više od 10 žetona.

Rezervisanjem karte igrač sprečava protivnika da je kupi i dobija zlatni žeton, koji može zameniti bilo koju boju. Igrač može imati najviše tri rezervisane karte.

Kupovinom karte igrač plaća cenu žetonima, umanjenu za trajne bonuse koje već ima. Karte mogu donositi prestižne poene i uvek daju trajni bonus određene boje.

Plemići donose dodatne poene kada igrač sakupi dovoljan broj razvojnih karata određenih boja. Igra se završava kada neki igrač dostigne najmanje 15 poena, nakon čega se završava poslednja runda.

Za igru sa 2 igrača koristi se 4 žetona svake osnovne boje, svih 5 zlatnih žetona i 3 plemića. Na tabli je 12 vidljivih karata, po 4 karte za svaki nivo.

3. Postojeća rešenja

Postoje **strategijski vodiči** i forumi koji daju savete za igru Splendor, kao i **AI agenti** koji mogu igrati kao protivnik. Primeri:

- https://en.doc.boardgamearena.com/Tips_splendor
- <https://boardgamegeek.com/thread/1722407/quick-win-strategies>
- https://www.reddit.com/r/boardgames/comments/knm3f/splendor_whats_your_strategy/
- <https://www.diva-portal.org/smash/get/diva2:1779339/FULLTEXT01.pdf>

Predloženi sistem se razlikuje od njih jer **koristi bazu pravila, zasnovanu na strategiji, za analizu stanja igre, izvođenje strateških ciljeva i bodovanje konkretnih poteza**. Na osnovu toga daje listu rangiranih poteza i objašnjenje načina ocenjivanja.

4. Metodologija rada

Sistem se sastoji iz sledećih celina:

- baza činjenica,
- baza pravila,
- forward chaining mehanizam,
- accumulate funkcija za bodovanje poteza,
- backward chaining za proveru uticaja na odluku

5. Ulaz i izlaz iz sistema

Ulaz u sistem

Ulaz predstavlja **opis trenutnog stanja** partije:

- trenutno stanje igrača
- trenutno stanje protivnika
- trenutno stanje table

Ovi podaci su modelovani na sledeći način:

Player:

- prestigePoints
- tokens[color]
- goldTokens
- bonuses[color]
- reservedCards
- purchasedCards
- totalTokens

Opponent:

- prestigePoints
- tokens[color]
- goldTokens
- bonuses[color]
- reservedCards
- purchasedCards
- totalTokens

Board:

- visibleCards[level]
- nobles
- bankTokens[color]
- bankGoldTokens

Izlaz iz sistema

Izlaz predstavlja **rangiranu listu legalnih poteza**. Sistem prikazuje:

- preporučeni potez
- rangiranu listu legalnih poteza sa skorovima

Nakon izbora nekog poteza:

- ukupan skor
- pozitivne i negativne razloge bodovanja
- listu uticaja tog poteza
- stablo bodovanja

6. Baza znanja

Baza znanja sadrži podatke o **razvojnim kartama, plemićima, mogućim akcijama i pravilima** za procenu poteza.

Razvojne karte se modeluju kroz:

Card:

- id
- level
- colorBonus
- prestigePoints
- cost[color]
- totalCost

Plemići se modeluju kroz:

Noble:

- requiredBonuses[color]
- prestigePoints

Moguće akcije su:

- uzimanje tri različita žetona
- uzimanje dva ista žetona ako u banci ima bar 4 žetona te boje
- rezervacija dostupne karte ako igrač ima manje od 3 rezervisane karte
- kupovina dostupne karte
- kupovina rezervisane karte

Potezi se modeluju kroz:

Move:

- type = BUY_CARD / RESERVE_CARD / TAKE_TOKENS
- card
- score
- takenTokens[color]

Baza znanja sadrži pravila za određivanje efikasnosti karte, dominantne boje, strateških ciljeva i bodovanje poteza.

Forward chaining

Sistem preporučuje poteze prema **High efficiency strategiji**, koja se zasniva na izboru karata koje donose najveću korist za najmanju cenu.

Pravila su organizovana u tri nivoa forward chaining zaključivanja:

- **Nivo 1 - analiza trenutnog stanja igre**
- **Nivo 2 - izvođenje strateških ciljeva**
- **Nivo 3 - bodovanje legalnih poteza**

Na kraju se koristi accumulate funkcija koja sabira poene svih aktiviranih pravila za svaki od poteza.

Nivo 1 - Analiza stanja igre

Izlaz:

- gamePhase = EARLY_GAME / MID_GAME / LATE_GAME
- cardEfficiency(card) = VERY_HIGH / HIGH / LOW
- dominantColor(color)
- purchasableCard(card)
- almostPurchasableCard(card)
- opponentThreatCard(card)
- neededColor(color)
- supportsEfficiencyEngine(card)
- tokenLimitRisk

Pravila:

1. Rana faza igre

IF player.prestigePoints < 5
THEN gamePhase = EARLY_GAME

2. Srednja faza igre

IF player.prestigePoints >= 5
AND player.prestigePoints < 12
THEN gamePhase = MID_GAME

3. Kasna faza igre

IF player.prestigePoints >= 12
THEN gamePhase = LATE_GAME

4. Veoma efikasna karta

IF card.prestigePoints >= 3
AND card.totalCost <= 7
THEN cardEfficiency(card) = VERY_HIGH

5. Efikasna karta

IF card.prestigePoints >= 1
AND card.totalCost <= 5
THEN cardEfficiency(card) = HIGH

6. Neefikasna karta

IF card.totalCost > 8
AND card.prestigePoints <= 2
THEN cardEfficiency(card) = LOW

7. Dominantna boja

IF numberOfEfficientCardsWithBonus(color) >= 2
OR numberOfEfficientCardsThatRequire(color) >= 2
THEN dominantColor(color)

8. Karta je odmah kupiva

IF player can pay card cost using tokens, bonuses and gold
THEN purchasableCard(card)

9. Karta je skoro kupiva

IF missingTokensForCard(player, card) <= 2
THEN almostPurchasableCard(card)

10. Protivnikova opasna karta

IF missingTokensForCard(opponent, card) <= 2
AND cardEfficiency(card) = VERY_HIGH
THEN opponentThreatCard(card)

11. Potrebna boja za kupovinu efikasne karte

IF cardEfficiency(card) = VERY_HIGH
AND almostPurchasableCard(card)
AND player needs token color X to buy card
THEN neededColor(X)

12. Potrebna boja za kupovinu rezervisane karte

IF card IN player.reservedCards
AND missingTokensForCard(player, card) <= 3
AND player needs token color X to buy card
THEN neededColor(X)

13. Karta podržava high efficiency engine ako daje dominantnu boju

IF cardEfficiency(card) = VERY_HIGH
AND card.colorBonus = color
AND dominantColor(color)
THEN supportsEfficiencyEngine(card)

14. Karta podržava engine ako pomaže kupovinu druge veoma efikasne karte

IF cardEfficiency(card) = HIGH
AND card.colorBonus = color
AND exists targetCard where cardEfficiency(targetCard) = VERY_HIGH
AND targetCard.cost[color] > 0
THEN supportsEfficiencyEngine(card)

15. Rezervisana karta podržava engine ako se može uskoro odigrati

IF card IN player.reservedCards
AND cardEfficiency(card) = VERY_HIGH
AND missingTokensForCard(player, card) <= 3
THEN supportsEfficiencyEngine(card)

16. Rizik limita žetona

IF player.totalTokens > 8
THEN tokenLimitRisk = true

Nivo 2 - Izvođenje strateških ciljeva

Nivo 2 koristi zaključke iz Nivoa 1 i na osnovu njih određuje strateške ciljeve.

Izlaz:

- strategicGoal = BUY_CARD(card)
- strategicGoal = RESERVE_CARD(card)
- strategicGoal = BUILD_DOMINANT_COLOR(color)
- strategicGoal = BLOCK_OPPONENT(card)
- strategicGoal = COLLECT_TOKENS(color)
- strategicGoal = PLAY_RESERVED_CARD(card)
- strategicGoal = MANAGE_TOKEN_LIMIT

Pravila:

17. Cilj je kupovina veoma efikasne karte koja podržava engine

IF purchasableCard(card)
AND cardEfficiency(card) = VERY_HIGH
AND supportsEfficiencyEngine(card)
THEN strategicGoal = BUY_CARD(card)

18. Cilj je kupovina efikasne karte u ranoj fazi

IF purchasableCard(card)
AND cardEfficiency(card) = HIGH
AND gamePhase = EARLY_GAME
AND supportsEfficiencyEngine(card)
THEN strategicGoal = BUY_CARD(card)

19. Cilj je rezervacija veoma efikasne karte koja se uklapa u plan

IF cardEfficiency(card) = VERY_HIGH
AND supportsEfficiencyEngine(card)
AND purchasableCard(card) = false
AND numberOfReservedCards(player) < 3
THEN strategicGoal = RESERVE_CARD(card)

20. Cilj je razvoj dominantne boje

IF dominantColor(color)
AND gamePhase != LATE_GAME
THEN strategicGoal = BUILD_DOMINANT_COLOR(color)

21. Cilj je blokiranje protivnika

IF opponentThreatCard(card)
AND numberOfReservedCards(player) < 3
THEN strategicGoal = BLOCK_OPPONENT(card)

22. Cilj je skupljanje potrebne boje

IF neededColor(color)
AND bankTokens(color) > 0

THEN strategicGoal = COLLECT_TOKENS(color)

23. Cilj je igranje rezervisane karte

IF card IN player.reservedCards
AND purchasableCard(card)
THEN strategicGoal = PLAY_RESERVED_CARD(card)

24. Cilj je upravljanje limitom žetona

IF tokenLimitRisk = true
THEN strategicGoal = MANAGE_TOKEN_LIMIT

Nivo 3 - Bodovanje poteza

Na trećem nivou sistem prvo **generiše skup legalnih poteza** na osnovu trenutnog stanja table, dostupnih žetona u banci, rezervisanih karata i mogućnosti plaćanja. Nakon toga **svaki potez se boduje** na osnovu prethodno izvedenih zaključaka i njegovog doprinosa izabranoj strategiji.

Izlaz:

- moveScore(move)
- rankedMoves
- recommendedMove

Pravila:

25. Kupovina veoma efikasne karte

IF move.type = BUY_CARD
AND move.card = card
AND strategicGoal = BUY_CARD(card)
THEN moveScore(move) += 40

26. Kupovina karte koja razvija dominantnu boju

IF move.type = BUY_CARD
AND move.card.colorBonus = color
AND strategicGoal = BUILD_DOMINANT_COLOR(color)
THEN moveScore(move) += 20

27. Kupovina karte koja podržava high efficiency engine

IF move.type = BUY_CARD
AND move.card = card
AND supportsEfficiencyEngine(move.card)
THEN moveScore(move) += 20

28. Kupovina karte sa poenima u kasnoj fazi

IF move.type = BUY_CARD
AND gamePhase = LATE_GAME
AND move.card.prestigePoints > 0
THEN moveScore(move) += 30

29. Kupovina karte koja završava igru

IF move.type = BUY_CARD
AND player.prestigePoints + move.card.prestigePoints >= 15
THEN moveScore(move) += 150

30. Kupovina neefikasne karte se kažnjava

IF move.type = BUY_CARD
AND cardEfficiency(move.card) = LOW
AND gamePhase != LATE_GAME
THEN moveScore(move) -= 20

31. Rezervisanje veoma efikasne karte

IF move.type = RESERVE_CARD
AND move.card = card
AND strategicGoal = RESERVE_CARD(card)

THEN moveScore(move) += 35

32. Rezervisanje karte koja blokira protivnika

IF move.type = RESERVE_CARD
AND move.card = card
AND strategicGoal = BLOCK_OPPONENT(card)
THEN moveScore(move) += 30

33. Kupovina rezervisane karte

IF move.type = BUY_CARD
AND move.card = card
AND strategicGoal = PLAY_RESERVED_CARD(card)
THEN moveScore(move) += 35

34. Rezervisanje karte koja ne podržava engine se kažnjava

IF move.type = RESERVE_CARD
AND NOT supportsEfficiencyEngine(move.card)
AND NOT opponentThreatCard(move.card)
THEN moveScore(move) -= 20

35. Kažnjava se rezervacija koja može da zaglavi igrača

IF move.type = RESERVE_CARD
AND numberOfReservedCards(player) = 2
AND missingTokensForCard(player, move.card) > 4
THEN moveScore(move) -= 25

36. Uzimanje potrebne boje

IF move.type = TAKE_TOKENS
AND move.takenTokens contains color
AND strategicGoal = COLLECT_TOKENS(color)
THEN moveScore(move) += 15

37. Uzimanje dominantne boje

```
IF move.type = TAKE_TOKENS  
AND move.takenTokens contains color  
AND dominantColor(color)  
THEN moveScore(move) += 10
```

38. Uzimanje nepotrebnih žetona se kažnjava

```
IF move.type = TAKE_TOKENS  
AND move.takenTokens does not contain any color C  
  for which strategicGoal = COLLECT_TOKENS(C)  
THEN moveScore(move) -= 20
```

39. Kupovina karte smanjuje rizik limita žetona

```
IF move.type = BUY_CARD  
AND strategicGoal = MANAGE_TOKEN_LIMIT  
THEN moveScore(move) += 15
```

40. Uzimanje žetona preko limita se kažnjava

```
IF move.type = TAKE_TOKENS  
AND strategicGoal = MANAGE_TOKEN_LIMIT  
AND player.totalTokens + numberOfTakenTokens(move) > 10  
THEN moveScore(move) -= 25
```

Accumulate funkcija

Pravila trećeg nivoa kreiraju **MoveScoreFact** činjenice kojima se potezu dodeljuje određeni broj poena. Svaka MoveScoreFact činjenica sadrži **potez, broj poena i razlog** bodovanja.

Na kraju se koristi Drools **accumulate pravilo** koje za svaki legalan potez pronalazi sve njegove MoveScoreFact činjenice i **sabira njihove poene**. Tako se dobija ukupan skor poteza koji se koristi pri rangiranju svih legalnih poteza. Preporučeni potez je potez sa najvećim ukupnim skorom.

Meta činjenice

Pored domenskih činjenica koje predstavljaju stanje igre i izvedene zaključke, sistem tokom izvršavanja pravila kreira i meta činjenice tipa **DecisionFact**. Ove činjenice ne učestvuju direktno u bodovanju poteza, već služe **za objašnjenje zaključivanja**.

Svaka DecisionFact činjenica povezuje jedan **izvedeni zaključak sa činjenicom** ili zaključkom koji je doveo do njega. Sadrži sledeće podatke:

- zaključak,
- preduslov,
- nivo zaključivanja,
- objašnjenje

Na primer, ako pravilo zaključi da je karta veoma efikasna, sistem pored CardEfficiency činjenice kreira i DecisionFact koji povezuje taj zaključak sa početnom činjenicom o broju poena i ukupnoj ceni karte.

Primer:

- Card RES-WHITE-3 has at least 3 points and total cost at most 7
→ cardEfficiency(RES-WHITE-3) = VERY_HIGH

se beleži kao:

DecisionFact(

conclusion = analysis:CARD_EFFICIENCY:VERY_HIGH:RES-WHITE-3,

prerequisite = base:RES-WHITE-3 has at least 3 points and total cost at most 7,

level = ANALYSIS

)

Na isti način se povezuju i strateški ciljevi i bodovanje poteza. Time se tokom forward chaininga gradi **graf zavisnosti** između početnih činjenica, izvedenih analiza, strateških ciljeva i dodeljenih poena. Taj graf se kasnije koristi u backward chaining query-ju za **proveru i objašnjenje uticaja pojedinačnih činjenica** na izabrani potez.

Backward chaining

Backward chaining se realizuje **nad DecisionFact meta činjenicama**. Upit **explainsDecision(conclusion, prerequisite)** rekurzivno proverava da li zaključak direktno ili indirektno zavisi od neke prethodne činjenice.

Logika query-ja je:

- conclusion zavisi od prerequisite

ILI

- conclusion zavisi od intermediate zaključka, a intermediate rekurzivno zavisi od prerequisite.

Na taj način sistem može da proveriti da li je izabrani potez zavisio od određene činjenice. Na primer od konkretne analize ili strateškog cilja. Pomoću backward chaininga se kreira **odgovor na pitanja iz liste uticaja poteza**:

1. Da li je izabrana karta veoma efikasna?
2. Da li je potez kažnjen zbog niske efikasnosti?
3. Da li izabrana karta podržava engine?
4. Da li potez razvija dominantnu boju?
5. Da li potez skuplja potrebne tokene?
6. Da li potez blokira protivnika?
7. Da li potez dovodi do problema sa limitom tokena?

Template

Template mehanizam se koristi u trećem nivou zaključivanja, za generisanje sličnih **pravila za bodovanje poteza**. Pošto različiti tipovi poteza koriste različite uslove, definisana su tri template-a:

1. ScoreBuyCardMove
2. ScoreReserveCardMove
3. ScoreTakeTokensMove

Na primer ScoreBuyCardMove se koristi za pravila koja boduju kupovinu karte:

```
IF move.type = BUY_CARD  
  
AND move.card = card  
  
AND {condition}  
  
THEN moveScore(move) += {points}
```

Tabela za popunjavanje ScoreBuyCardMove template-a:

condition	points
strategicGoal = BUY_CARD(card)	+40
supportsEfficiencyEngine(card)	+20
gamePhase = LATE_GAME AND card.prestigePoints > 0	+30
cardEfficiency(card) = LOW AND gamePhase != LATE_GAME	-20
strategicGoal = MANAGE_TOKEN_LIMIT	+15